

**THE UNITED REPUBLIC OF TANZANIA
PRIME MINISTER'S OFFICE
REGIONAL ADMINISTRATION AND LOCAL GOVERNMENT
NJOMBE REGION
FORM SIX PRE-MOCK EXAMINATION**

136/2

COMPUTER SCIENCE 2

(For Both School and Private Candidate)

Time 3 Hours

Thursday, 28th August 2026 a.m

Instructions

1. This paper consists of **Three** (3) questions
2. Answer **two** (2) questions, including question number **one** (1).
3. Save your work on the desktop in the folder named by your Examination Number.
4. Save your work by using the 1997-2003 version of the MS Office Software.
5. Check whether the **Printed** work(s) are similar to the **softcopy** saved in the folder.
6. Submit printed codes and screenshots together with the softcopy of your Work(s).
7. Any communication media is not allowed in the examination room.
8. Write your name on every paper/answer sheet you have used.

wazaelimu.com

QUESTION 1

(a) The Headmaster of a secondary school in Mwanza requires a flexible computer application to compute total marks for students across various academic subjects. Different subjects have varying numbers of continuous assessments. For instance, some subjects only assess students via a Terminal Examination, whereas others combine Mid-Term test scores, project marks, and Terminal Examination results.

You are required to write an object-oriented Python program that utilizes variable-length arguments (*args) to calculate total marks regardless of the number of assessment scores passed.

Tasks:

Write a complete, well-commented Python program adhering to the following specifications:

1. Create a class named AcademicRecord.
2. Inside the class, define a method named calculate_total_marks.
3. Configure the method to accept any number of numerical scores using variable-length positional arguments (*args).
4. Compute and return the sum of all scores passed to the method. If no arguments are provided, the method must return 0.
5. In the main section of your script:
 - Instantiate an object of the AcademicRecord class.
 - **Case 1:** Invoke the method for a student who only has a Terminal Examination score (**75** marks).
 - **Case 2:** Invoke the method for a student who has a Mid-Term score (**15** marks) and a Terminal Examination score (**60** marks).
 - **Case 3:** Invoke the method for a student who has a Quiz (**10** marks), a Project score (**20** marks), and a Terminal Examination score (**55** marks).
6. Print all calculated total marks clearly to the console output.

QUESTION 2

Kilimani High School wishes to digitize its student club registration process. Previously, students registered manually using paper forms, leading to lost records and processing delays. As a Web Developer, you are tasked with designing and developing an interactive web-based portal titled "**Kilimani Club Portal**".

Technical Requirements:

- Save all source files (index.php/index.html, style.css, script.js, and process.php) within a root directory folder named using your **Examination Number** (e.g., S0202_0502) inside your local server directory (htdocs or www).

PART A: Front-End Form Structure & Styling (HTML & CSS)

1. HTML Web Page (index.php or index.html):

- Structure the document using semantic HTML5 elements (<header>, <main>, <footer>).
- Include an HTML form titled "**Club Membership Form**" configured to send data to process.php via the POST method. Include the following inputs:
 - **Full Name:** Single-line text input.
 - **Class Combination:** Dropdown menu (<select>) containing options: PCM, PCB, PGM, EGM, CBG, HGL.
 - **Preferred Club:** Radio button selection for: Science & Tech, Environmental, Debate.
 - **Phone Number:** Single-line text input.
 - **Controls:** A submit button labeled "**Register**" and a reset button labeled "**Clear**".

2. CSS Styling (style.css):

- Create an external stylesheet named style.css and link it to your web page.
- Apply a dark green background color to the header, a light grey background to the page body, and center the form inside a styled white container card.
- Apply consistent padding, subtle border radius, and full container width (100%) to all form inputs.
- Define smooth visual transition effects on the "**Register**" button upon mouse hover.

PART B: Client-Side Input Validation (JavaScript)

3. Validation Script (script.js):

- Create an external JavaScript file named script.js to perform client-side form validation prior to submission:
 - Verify that the **Full Name** input field contains at least two distinct words separated by a space.
 - Verify that the **Phone Number** starts with the digit 0 and contains exactly 10 numeric digits.
 - If validation fails, prevent form submission and display a dynamic, clear error message in **red text** below or beside the corresponding field without reloading the page.

- If validation passes, display a browser alert pop-up with the message: "Validation successful! Submitting your details..."

PART C: Server-Side Processing & Output (PHP)

4. Server-Side Script (process.php):

- Create a PHP script named process.php to handle incoming data via the HTTP POST method.
- Sanitize and extract all input data sent from the form.
- Validate server-side that no mandatory fields are left blank.
- Display a clean styled confirmation summary card presenting the student's registration details in the exact layout shown below:

Reservation Confirmation Summary

Student Name: [Submitted Name]

Class: [Form V / Form VI]

Reserved Book: [Submitted Book Title]

Date of Reservation: [Submitted Date]

Status: *Reservation Pending Approval*

Deliverables and Submission

1. Ensure your local server (e.g., XAMPP, WAMP, or LAMP) is running.
2. Test the complete application flow in a web browser to confirm client-side validation and PHP script execution.
3. Keep all files (index.php, style.css, script.js, and process.php) organised in your assigned examination folder.

QUESTION 3

The Kilimanjaro Agricultural Cooperative Society (KACS) collects produce from local farmers across different districts. To improve operational efficiency, management is transitioning from a manual bookkeeping system to a Relational Database Management System (RDBMS).

As a Database Administrator, design and implement a relational database to manage farmer supply operations.

Tasks & Requirements:

- (a) Create a database named **KACS_Database** inside your examination folder. (02 Marks)
- (b) Design two related tables strictly matching the field structures and property specifications provided below: (06 Marks)

Table 1: tblFarmers

Field Name	Data Type	Field Size / Format	Attributes / Rules
FarmerID	Short Text	Field Size: 6	Primary Key , Input Mask: \F-000
FullName	Short Text	Field Size: 50	Required: Yes
District	Short Text	Field Size: 25	—
PhoneNumber	Short Text	Field Size: 10	—
RegistrationDate	Date/Time	Short Date	—

Table 2: tblSupplies

Field Name	Data Type	Field Size / Format	Attributes / Rules
SupplyID	AutoNumber	—	Primary Key
FarmerID	Short Text	Field Size: 6	Foreign Key
CropType	Short Text	Lookup Wizard	Options: "Coffee", "Maize", "Beans", "Wheat"
Quantity_KG	Number	Double	Validation Rule: >0 Validation Text: "Quantity must be greater than zero!"
UnitPrice_TZS	Currency	Standard	—
SupplyDate	Date/Time	Short Date	Default Value: =Date()

(c) Establish a **One-to-Many** relationship between tblFarmers (FarmerID) and tblSupplies (FarmerID), and enable **Enforce Referential Integrity**. (02 Marks)

(d) Populate both database tables with the following records: (03 Marks)

Data for tblFarmers

FarmerID	FullName	District	PhoneNumber	RegistrationDate
F-101	Juma Bakari	Moshi Urban	0754123456	15/01/2023
F-102	Neema Swai	Hai	0784987654	20/02/2023
F-103	Baraka Mushi	Rombo	0713555888	10/03/2023
F-104	Asha Rashid	Siha	0767112233	05/04/2023

Data for tblSupplies

Supply ID	Farmer ID	CropType	Quantity_KG	UnitPrice_TZS	SupplyDate
1	F-101	Coffee	450	4500	12/05/2023
2	F-102	Maize	1200	800	18/05/2023
3	F-101	Coffee	300	4500	02/06/2023
4	F-103	Beans	650	2200	15/06/2023
5	F-104	Coffee	500	4800	20/06/2023
6	F-102	Wheat	850	1500	01/07/2023

(e) Construct the following database queries:

1. **qryCoffeeSupplies**: A select query returning the FullName, District, CropType, Quantity_KG, and UnitPrice_TZS for all supply entries where the CropType is "**Coffee**". (03 Marks)
2. **qryTotalPayout**: A query containing FullName, CropType, Quantity_KG, UnitPrice_TZS, and a calculated field named TotalPayout defined as:

$$\text{TotalPayout} = \text{Quantity_KG} \times \text{UnitPrice_TZS}$$

(03 Marks)

(f) Create a Data Entry Form named **frmFarmerEntry** based on tblFarmers:

- Include a main form displaying farmer information alongside an embedded **Subform** displaying all matching supply records from tblSupplies.
- Add interactive Command Buttons to "**Add New Record**" and "**Save Record**". (03 Marks)

(g) Generate a Database Report named **rptSupplySummary** based on qryTotalPayout:

- Group all records by **CropType**.
- Display the **Sum of TotalPayout** within each crop grouping footer.
- Set the report header title to "**KACS FARMERS PRODUCE SUPPLY REPORT**". (03 Marks)